线性数据结构与DP优化

利用栈和队列的单调性质，优化枚举区间内极值$O(n)$的时间复杂度到$O(1)$

问题的特征：

1. 区间内的极值(最大值，最小值)
2. 找当前数左(右)边第一个比当前数大(小)的数

单调栈

P5788 【模板】单调栈

题目描述：给出项数为 n 的整数数列 $a_1,\dots,a_n$。

定义函数 $f(i)$代表数列中第 i 个元素之后第一个大于 a_i 的元素的**下标**，即 $f(i)=\min_{i<j\le n,a_j>a_i}\{j\}$。若不存在，则$f(i)=0$。

试求出$f(1\dots n)$。

输入输出样例：

输入#1	输出#1
5 1 4 2 3 5	2 5 4 5 0

算法分析：

本题其实就是查找每个数后面第一个比它**严格大**的数。本题是模板题的转换题。

我们对样例进行模拟可得以下表格。

下标	1	2	3	4	5
元素值	1	4	2	3	5
$f(i)$	2	5	4	5	0

从模拟过程中，每计算一个$f(i)$的时间复杂度接近$O(n)$， n个$f(i)$的查询，总的时间复杂度为$O(n^2)$。

```
for(int i=1;i<=n;i++) {
    for(int j=i+1;j<=n;j++) {
        if(a[j]>a[i]) {
            cout<<j<<" ";
            break;
        }
        else    cout<<0<<" ";   //如果没找到就输出0
    }
}
```

而通过单调栈的优化，我们可以把$f(i)$的查询优化到接近$O(1)$，n个$f(i)$的查询，总的时间复杂度为$O(n)$。接下来我们来看下具体的优化思路：我们观察可以发现，递减的元素2，3 不可能是答案。因为如果寻找右边第一个大的数，那么这个数一定是最前面的 4 。如果数据再大一点，我们可以简单得到结论，答案的元素值是呈现**单调性**。而元素的压入弹出是**后进先出**的，因此可以采用栈的数据结构来处理，因此该类问题也就是单调栈优化问题。

讲本题之前，我们先来看一道单调栈的另外一种问法：查找每个数左边第一个比它**严格小**的数。思路如下：

下标	1	2	3	4	5
元素值	1	4	2	3	5
$f(i)$	0	1	1	3	4

手写一个栈结构$s[\ ]$。初始化第一个元素为 1，栈顶指针为$tt=1$。目的维护一个单调递增结构。

- 当栈不为空时：
    - 令$x=a[2]=4$，当$x<=s[tt]$时，$tt--$。输出tt，压进4, 此时的栈为$1,4$
    - 令$x=a[3]=2$，当$x<=s[tt]$时，$tt--$。输出tt，压进2, 此时的栈为$1,2$
    - 令$x=a[4]=3$，当$x<=s[tt]$时，$tt--$。输出tt，压进3, 此时的栈为$1,2,3$
    - 令$x=a[5]=5$，当$x<=s[tt]$时，$tt--$。输出tt，压进5, 此时的栈为$1,2,3,5$

  输出tt的过程其实就是输出对应编号的过程，代码如下：

```
int s[N], tt=0;
for(int i=1;i<=n;i++) {
    int x;  cin>>x;
    while(x<=s[tt] && tt>0) tt--;    //输入一个数，维护栈的单调性。
    cout<<tt;        // 输出左边第一个严格小的数的编号
    s[++tt] = x;     //维护单调性。
}
```

接着回到本题的核心问题：查找每个数后面第一个比它**严格大**的数，如何将其转换为：查找每个数左边第一个比它**严格小**的数。

查找每个数左边第一个比它**严格小**的数。我们处理数据时，是从$1-n$开始。那么如果处理数据从$n-1$开始呢？

代码参考：

```
#include<bits/stdc++.h>
using namespace std;
const int N=3000010;
int a[N];
int stk[N],t;   //注意：这里的栈stk是存储编号，不是数值

int f[N];
int main() {
    int n;  cin>>n;
    for(int i=1;i<=n;i++)   cin>>a[i];
    for(int i=n;i;i--) {
        while(t>0 && a[i]>=a[stk[t]])   t--;
        f[i]=stk[t];
        stk[++t]=i;
    }
```

```
    for(int i=1;i<=n;i++)    cout<<f[i]<<" ";
    return 0;
}
```

单调队列

P1886 滑动窗口 /【模板】单调队列

题目描述：有一个长为 n 的序列 a ，以及一个大小为 k 的窗口。现在这个从左边开始向右滑动，每次滑动一个单位，求出每次滑动后窗口中的最大值和最小值。

窗口元素	最大值	最小值
$[1,3,-1],-3,5,3,6,7$	3	-1
$1,[3,-1,-3],5,3,6,7$	3	-3
$1,3,[-1,-3,5],3,6,7$	5	-3
$1,3,-1,[-3,5,3],6,7$	5	-3
$1,3,-1,-3,[5,3,6],7$	6	3
$1,3,-1,-3,5,[3,6,7]$	7	3

算法分析：

本题如果我们采用直接暴力求解区间最大值最小值，时间复杂度是$O(k)$。算上$(n-k+1)$个区间，总的时间复杂度接近$O(nk)$，会超时。这里我们发现，如果能够把暴力求解区间最值优化到$O(1)$，那么总时间复杂度就可以接受了。 我们创建一个队列，维护队列里的元素呈单调递减性。队首即为最大值，队列长度不超过$k=3$。

我们来模拟一下过程：

- 压入当前元素 $x=1$，此时队列为$\{1\}$， 队首为1
- 压入当前元素$x=3$，当x大于队首时，弹出继续维护单调性，此时队列为$\{3\}$， 队首为3
- 压入当前元素$x=-1$，当x大于队首时，弹出继续维护单调性，此时队列为$\{3,-1\}$， 队首为3
- 压入当前元素$x=-3$，当x大于队首时，弹出继续维护单调性，此时队列为$\{3,-1,-3\}$ ， 队首为3
- 压入当前元素$x=5$，当x大于队首时，弹出继续维护单调性，此时队列为$\{5\}$， 队首为5
- 压入当前元素$x=3$，当x大于队首时，弹出继续维护单调性，此时队列为$\{5,3\}$， 队首为5
- 压入当前元素$x=6$，当x大于队首时，弹出继续维护单调性，此时队列为$\{6\}$， 队首为6
- 压入当前元素$x=7$，当x大于队首时，弹出继续维护单调性，此时队列为$\{7\}$， 队首为7

我们接着分析：从k开始，队首元素值就是最大值。但是这样维护会有一个问题，就是队列长度有可能大于k。这个时候进行特判下，如果大于k，队首需要弹出。

代码参考：

```
#include<iostream>
#include<cstdio>
using namespace std;
const int N=1000010;
int a[N];
int p[N],q[N];  //定义两个队列存储数组的下标编号
int hh,tt;
```

```cpp
int main() {
    int n,k;
    cin>>n>>k;
    for(int i=1;i<=n;i++)   cin>>a[i];
    hh=1,tt=1;
    //维护单调递增队列
    for(int i=1;i<=n;i++){
        if(i-k+1 > p[hh])   ++hh;   //如果当前的数据超过窗口大小时，则队首右移
        while(hh<=tt && a[i]<=a[p[tt]]) --tt;   //如果当前数据不符合递增属性，则移动队尾指针
        p[++tt] = i;    //存储符合递增属性的元素
        if(i+1>k)   cout<<a[p[hh]]<<" ";    //如果达到窗口的大小，则输出队首
    }
    cout<<endl;
    //维护单调递减队列
    hh=0,tt=1;
    for(int i=1;i<=n;i++){
        if(i-k+1>q[hh]) ++hh;
        while(hh<=tt && a[i]>=a[q[tt]]) tt--;
        q[++tt]=i;
        if(i+1>k)   cout<<a[q[hh]]<<" ";
    }
    return 0;
}
```

例题推荐

- P1440 求m区间内的最小值
- P1638 逛画展
- P1714 切蛋糕
- P2032 扫描
- P3512 [POI2010]PIL-Pilots
- P1901 发射站
- P1823 [COI2007] Patrik 音乐会的等待

单调队列优化DP

P1776 宝物筛选

题目描述：小 F 对洞穴里的宝物进行了整理，他发现每样宝物都有一件或者多件。他粗略估算了下每样宝物的价值，之后开始了宝物筛选工作：小 F 有一个最大载重为 W 的采集车，洞穴里总共有 n 种宝物，每种宝物的价值为 v_i，重量为 w_i，每种宝物有 m_i 件。小 F 希望在采集车不超载的前提下，选择一些宝物装进采集车，使得它们的价值和最大。

输入输出格式：

第一行为一个整数 n 和 W，分别表示宝物种数和采集车的最大载重。

接下来 n 行每行三个整数 v_i, w_i, m_i。

输出仅一个整数，表示在采集车不超载的情况下收集的宝物的最大价值。

输入输出样例：

输入样例	输出样例
4 20 3 9 3 5 9 1 9 4 2 8 1 3	47

数据说明：

- 对于 30% 的数据，$n \le \sum m_i \le 10^4$，$0 \le W \le 10^3$。
- 对于 100% 的数据，$n \le \sum m_i \le 10^5$，$0 \le W \le 4 \times 10^4$，$1 \le n \le 100$。

算法分析

$30pts$ ： 时间复杂度是$O(n * W * \sum m_i)$

01多重背包的模板题。每件物品可以选择m_i次。

$100pts$：

单调队列优化DP其实就是去掉一些"无用"的转移过程。我们先来看下原始的DP方程式：

$f[i][j] = max(f[i-1][j], f[i-1][j-k*w[i]] + k*v[i])$

其中，$k = min(V/w[i], m[i])$，以下用lim 表示$min(V/w[i], m[i])$

容易发现的是$f[i][j-k*w[i]]$会被$f[i][j-(k+1)*w[i]]$ 影响。归纳来看的话：

$f[i][j]$将会影响$f[i][j+k*w[i]]$ $(j+k*w[i] <= W)$



容易发现的是,同一颜色的格子,对$w[i]$取模得到的余数相同。且它们的差满足等差数列（公差为$w[i]$）。**通项公式为：$j = k*w[i]$+余数**

令$d = w[i]$， $a = j/w[i]$， $b = j\%w[i]$

其中a为全选状况下的物品个数，则$j = a*d+b$，代入原始DP方程式可得：

$f[i][j] = max(f[i-1][j], f[i-1][(a*d+b) - k*w[i]] + k*v[i])$

我们令$(a-k) = k'$，而$d = w[i]$，代入上述方程式可化简为：

$f[i][j] = max(f[i-1][j], f[i-1][(k'd+b) - k'*v[i]] + a*v[i])$

而又因为$a*v[i]$是一个常量，故方程式可转化为：

$f[i][j] = max(f[i-1][j], f[i-1][(k'd+b) - k'*v[i]]) + a*v[i]$

从上述方程式我们可知：将前面所有的$f[i-1][(k'd+b) - k'*v[i]]$放入一个队列，问题就转化为：求这个最长为$lim+1$的队列的最大值+$a+v[i]$。

代码参考

$30pts$ ：

```
#include<bits/stdc++.h>
using namespace std;
const int N=10010, M=1010;
int f[M];
int v[N],w[N],m[N];
int n,W;
```

```cpp
int main() {
    cin>>n>>W;
    for(int i=1;i<=n;i++)   cin>>v[i]>>w[i]>>m[i];
    for(int i=1;i<=n;i++) {
        for(int j=1;j<=m[i];j++) {
            for(int k=W;k>=w[i];k--) {
                f[k]=max(f[k],f[k-w[i]]+v[i]);

            }
        }
    }
    cout<<f[W];
    return 0;
}
```

$100pts$：

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N=40010,M=100010;
struct node {
    int pos, val;
}que[N];
int f[M];
int main() {
    int n,W;
    int v,w,m;
    cin>>n>>W;
    for(int i=1;i<=n;i++) {
        cin>>v>>w>>m;
        if(W/w <m)  m=W/w;  //计算lim
        for(int mo=0;mo<w;mo++) {
            int head=0, tail=0;
            for(int k=0;k<=(W-mo)/w;k++) {  //这里的k是k'
                int x=k;
                int y=f[k*w+mo]-k*v;
                while(head<tail && que[head].pos<k-m)   head++; //限制
                while(head<tail && que[tail-1].val<=y)  tail--; //维护单调性
                que[tail].val=y;
                que[tail].pos=x;
                tail++;
                f[k*w+mo]=que[head].val+k*v;    //更新最值
            }
        }
    }
    cout<<f[W];
    return 0;
}
```

例题推荐

- P1776 宝物筛选
- P1725 琪露诺
- P3089 [USACO13NOV]Pogo-Cow
- P3572 [POI2014]PTA-Little Bird
- P3522 [POI2011]TEM-Temperature

- P4544 [USACO10NOV]Buying Feed

状态压缩DP

DP状态表示的不是一个有意义的数值（例如，花费、价值、长度等），而是代表了集合的数量。这种 ***处理复杂集合*** 问题的DP叫做状态压缩DP。

集合的状态有很多，操作复杂，往往把方案用二进制数（“压缩”到这个二进制数中）来表示和操作。二进制操作有与、或、取反、移位等。

P1879 [USACO06NOV]Corn Fields G

算法分析：

提示： 在样例中有9种方案。



分别是：$\{\}$、$\{1\}$、$\{2\}$、$\{3\}$、$\{4\}$、$\{1,3\}$、$\{1,4\}$、$\{3,4\}$、$\{1,3,4\}$

这个方格图共 $m \times n$ 个格子，有$2^{m \times n}$种排列，无法用暴力法计算。

用下面的方法编程计算，算法复杂度是$O(m2^n2^n)$。

- 1) 方格的表示

很容易想到，可以用二进制数来描述方格，1表示种玉米，0表示不种玉米。在样例中，第1行的3个方格都是肥沃的，排除相邻的情况，有以下5种种玉米的方案：

- 2) DP状态和状态转移

如何设计DP状态，把问题从小规模逐步扩展到大规模？可以按行进行扩展。

在样例中，第2行只有两种方案。

这里的编号并不多余，在下面设计DP状态的时候有用。

第1行	编号	1	2	3	4	5
	方案	000	001	010	100	101

第2行	编号	1	2
	方案	000	010

洛谷@e_thinker

如果第2行选编号1的000，第1行可以选5种方案而不冲突。

如果第2行选编号2的010，第1行的010有冲突，第1行的其他4种方案没问题。

共$5+4=9$种方案。

用$dp[i][j]$表示第i行采用第j种编号的方案时前i行可以得到的可行方案总数。例如，$dp[2][2]=4$表示第2行使用第2种方案（即010）时的方案总数是4。

从第$i-1$行转移到第i行，状态转移方程如下：

$$dp[i][k] = \sum_{j=1}^{n} dp[i-1][j]$$

洛谷@e_thinker

其中j是第$i-1$行可行方案的编号，而且所有的$dp[i-1][j]$与第i行不冲突。 把最后一行的$dp[m][k]$，$(1 \le k \le n)$相加就得到了答案。

代码参考：

```
#include<bits/stdc++.h>
using namespace std;
const int MOD=1000000000;
int m,n,top;         //top为 符合条件的初始方案数
int state[1<<10] , num[150];     //state[i]:  编号为i的方案是state[i]
int dp[15][1<<10];
int cur[15];     //当前行状态
bool check(int x) {      //检查当前行是否有相邻1
    if(x& x<<1) return false;
    return true;
}
void init() {   //初始化
    top=0;
    int total = 1<<n;
    for(int i=0;i<total;i++) {
        if(check(i))    state[++top]=i;
    }
}
bool fit(int x,int k) {      //当前方案与当前行是否符合，即不能在贫瘠土地上种玉米
    if(x&cur[k])    return false;   //比如：cur = 010(2),  x = 100(2),  二进制与运算
    return true;
}
int main() {
    cin>>m>>n;
    init();
    for(int i=1;i<=m;i++) {
        cur[i]=0;
        int num;
        for(int j=1;j<=n;j++) {
            cin>>num;
            if(num==0){
                cur[i]+=(1<<(n-j));      //细节:  将输入的0，1 取反并用十进制存储。
            }
        }
    }
    for(int i=1;i<=top;i++) {
        if(fit(state[i],1)) dp[1][i]=1;      //第一行上符合种地的方案初始为1
    }

    for(int i=2;i<=m;i++) {
        for(int k=1;k<=top;k++) {
            if(!fit(state[k],i))    continue;
            for(int j=1;j<=top;j++) {
                if(!fit(state[j],i-1))  continue;
                if(state[k]&state[j])   continue;
```

```
                dp[i][k]=(dp[i][k]+dp[i-1][j])%MOD;
            }
        }
    }
    int ans=0;
    for(int i=1;i<=top;i++) ans= (ans+dp[m][i])%MOD;
    cout<<ans;
    return 0;
}
```

例题推荐：

- P1776 宝物筛选
- P1896 [SCOI2005]互不侵犯
- P2704 [NOI2001] 炮兵阵地
- P2051 [AHOI2009]中国象棋
- P2831 [NOIP2016 提高组] 愤怒的小鸟